Exercise 3 – Question 4

AdaBoost on Circle Data (toy.txt)

AdaBoost is applied to the toy dataset for 100 iterations using an LDA base classifier. The dataset has two continuous features and binary labels (+1 / -1). The +1 class forms a ring-like pattern while the -1 class clusters near the origin, making it non-linearly separable by a single linear classifier.

Figures 1–3: Error Curves

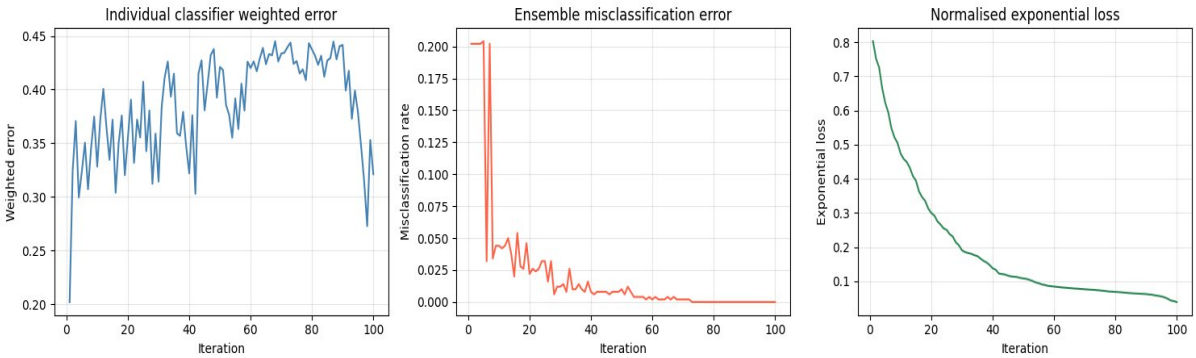

Fig 1 – Individual classifier weighted error. The weighted error of each LDA base classifier fluctuates between roughly 0.20 and 0.45 across iterations. It starts low because the initial uniform weights allow an easy split, then rises as AdaBoost concentrates weight on harder samples. Values staying well below 0.5 confirm that each weak learner is still better than random on its re-weighted distribution.

Fig 2 – Ensemble misclassification error. The ensemble error drops sharply in the first ~5 iterations from about 0.20 to near zero, with small spikes early on as the weighting adjusts. By iteration 40 it reaches essentially 0, demonstrating that boosting successfully combines weak classifiers into a strong one even for a non-linearly separable dataset.

Fig 3 – Normalised exponential loss. The exponential loss decreases monotonically from ~0.80 to below 0.10, consistent with AdaBoost's theoretical guarantee of minimising this loss. It decays faster than the 0/1 error and never increases, reflecting that the ensemble margin keeps growing even after misclassification rate plateaus.

Figures 4–7: Scatter Plots at Selected Iterations

Blue circles = correctly classified +1 points. Red squares = correctly classified -1 points. Cross markers (×) = misclassified points.

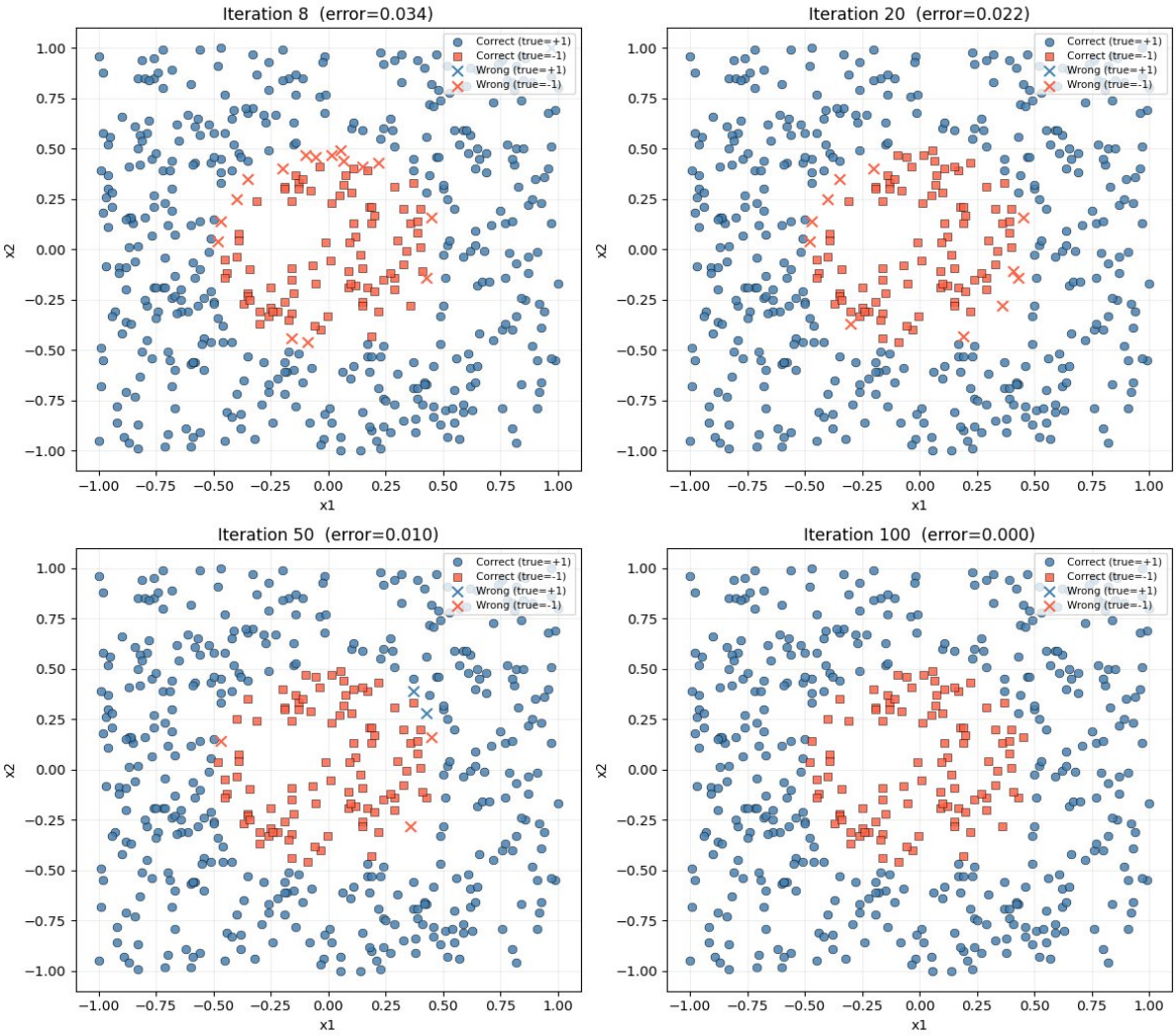

Fig 4 – Iteration 8 (error = 0.034). Already after just 8 iterations the ensemble classifies most points correctly. Errors concentrate on -1 points near the boundary with the +1 ring, where a linear classifier has difficulty.

Fig 5 – Iteration 20 (error = 0.022). Further improvement is visible. The number of misclassified -1 points decreases, though a few boundary cases remain wrong.

Fig 6 – Iteration 50 (error = 0.010). Only a handful of points near the class boundary are still misclassified. The ensemble has largely learned the non-linear structure through boosting.

Fig 7 – Iteration 100 (error = 0.000). By 100 iterations the ensemble achieves zero training error, correctly classifying all points. This confirms that AdaBoost can represent complex decision boundaries by combining many simple linear classifiers.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from adaboost import adaboost

# Load data
D = np.loadtxt('toy.txt')
labels = D[:, 0].copy()
D[:, 0] = 1         # replace label column with bias term (same as main())
X = D
y = labels
X_xy = D[:, 1:]     # original 2-D coords for scatter plots

# Run AdaBoost for 100 iterations
output, err_ind, err_ens, err_exp = adaboost(X, y, 100)
iters = np.arange(1, 101)

# Figures 1-3: Error curves
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].plot(iters, err_ind, color='steelblue', linewidth=1.5)
axes[0].set_title('Individual classifier weighted error')
axes[0].set_xlabel('Iteration')
axes[0].set_ylabel('Weighted error')
axes[0].grid(True, alpha=0.3)

axes[1].plot(iters, err_ens, color='tomato', linewidth=1.5)
axes[1].set_title('Ensemble misclassification error')
axes[1].set_xlabel('Iteration')
axes[1].set_ylabel('Misclassification rate')
axes[1].grid(True, alpha=0.3)

axes[2].plot(iters, err_exp, color='seagreen', linewidth=1.5)
axes[2].set_title('Normalised exponential loss')
axes[2].set_xlabel('Iteration')
axes[2].set_ylabel('Exponential loss')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Figures 4-7: Scatter plots at 8, 20, 50, 100 iterations
scatter_iters = [8, 20, 50, 100]

fig, axes = plt.subplots(2, 2, figsize=(12, 11))
axes = axes.flatten()

for ax, it in zip(axes, scatter_iters):
    out_t, _, _, _ = adaboost(X, y, it)
    pred = np.sign(out_t)

    correct   = pred == y
    incorrect = ~correct

    # Correctly classified points: circles (+1) and squares (-1)
    for label_val, marker in [(1, 'o'), (-1, 's')]:
        mask = correct & (y == label_val)
        color = 'steelblue' if label_val == 1 else 'tomato'
        ax.scatter(X_xy[mask, 0], X_xy[mask, 1],
                   marker=marker, c=color, edgecolors='k',
                   linewidths=0.4, s=40, alpha=0.85,
                   label=f'Correct (true={label_val:+d})')

    # Misclassified points: cross markers
    for label_val in [1, -1]:
        mask = incorrect & (y == label_val)
        color = 'steelblue' if label_val == 1 else 'tomato'
        ax.scatter(X_xy[mask, 0], X_xy[mask, 1],
                   marker='x', c=color, s=60, linewidths=1.5,
                   label=f'Wrong (true={label_val:+d})')
```

```
    err_rate = np.mean(pred != y)
    ax.set_title(f'Iteration {it}  (error={err_rate:.3f})')
    ax.set_xlabel('x1')
    ax.set_ylabel('x2')
    ax.legend(fontsize=7.5, loc='upper right')
    ax.grid(True, alpha=0.2)

plt.tight_layout()
plt.show()
```